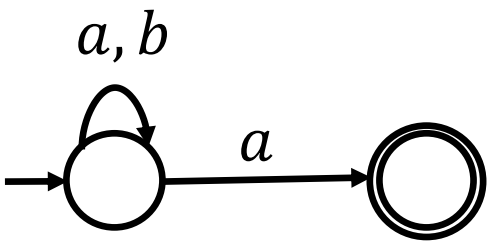
02141 – Computer Science Modelling

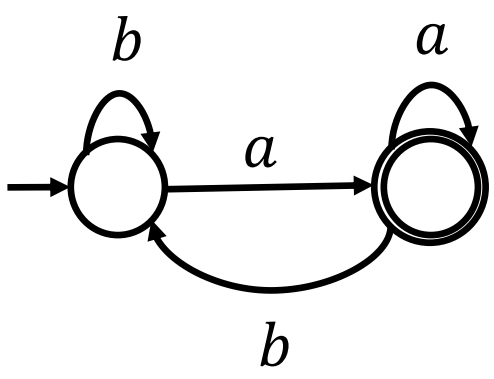
Based on the slides by Christoph Matheja

Last week: finite automata + non-determinism

NFA A



DFA A



- Is the language empty?
- Is the word $ababa$ accepted by the automaton?
- What is the language of A (or A)?

Why are we doing this? **Recap**

Theoretical computer science ≈ study of mathematical formalisms for languages

- What **languages** can be described by a **formalism**? What problems can a computer solve?
- For which problems and classes of languages there are **(efficient) algorithms**?

This course: *common formalisms and related problems*

Why are we doing this? **Recap**

Examples (languages)**:**

- ε, 0, 1, 111, 10101001 are words over the alphabet $\{0,1\}$
- $\{0, 11, 00101, 100\}$ is a language over the alphabet $\{0,1\}$
- *"set of all computer programs that eventually stop on all inputs"* is a language over $\{0,1\}$

Membership problem: does a word belong to a language?

- For regular languages described by DFAs, we can efficiently solve membership

For *arbitrary computer programs*, membership cannot be solved at all!

Formal Recap: Words

Alphabet Σ: finite set of *symbols*, e.g. $a, b, c, 0, 1, \ldots$

The set of words over Σ is defined inductively as follows:

- The empty word ε is a word over Σ
- Every symbol a in Σ is a word over Σ
- If u is a word over Σ and $a \in \Sigma$, then $a\,u$ and $u\,a$ are words over Σ

The length of a word w is given by $|w| = \begin{cases} 0, & \text{if } w = \varepsilon \\ 1 + |u|, & \text{if } w = a\,u \text{ and } a \in \Sigma \end{cases}$

The concatenation of v and w is $v \cdot w = v\,w = \begin{cases} v, & \text{if } w = \varepsilon \\ (v\,a) \cdot u, & \text{if } w = a\,u \text{ and } a \in \Sigma \end{cases}$

For $v = a_1 \ldots a_n$ and $w = b_1 \ldots b_m$, we have $v \cdot w = a_1 \ldots a_n\, b_1 \ldots b_m$

Formal Recap: Languages

Language L: set of words over Σ

$L_1\ L_2$: concatenation of L_1 and L_2

$$L_1 L_2 = L_1 \cdot L_2 = \{\, u\, v \mid u \in L_1, v \in L_2 \}$$

L^k: k concatenated copies of L

- $L^0 = \{\, \varepsilon \,\}$
- $L^{k+1} = L\, L^k$

Kleene closure of L: $L^* = \bigcup_{\{k \geq 0\}} L^k$

Examples over $\Sigma = \{a, b\}$:

$L_1 = \{ab\}$
$L_2 = \{ababab\}$
$L_3 = \{b, ab, aab, aaab, aaaab, \ldots\}$

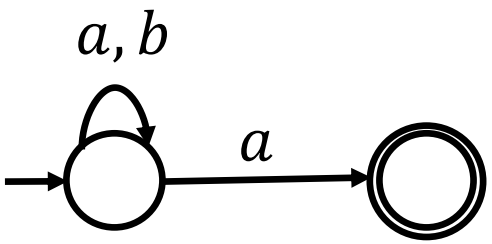
$L_2 = L_1^3 = L_1\ L_1\ L_1$

$L_3 = \{a\}^*\ \{\, b \,\}$

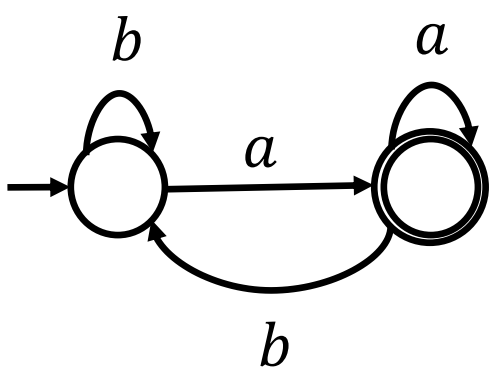
What languages are given by $\emptyset^0$ $\emptyset^3$ $\emptyset^*$ $\{\varepsilon\}^0$ $\{\varepsilon\}^5$ $\{\varepsilon\}^*$?

Back to the example

NFA A

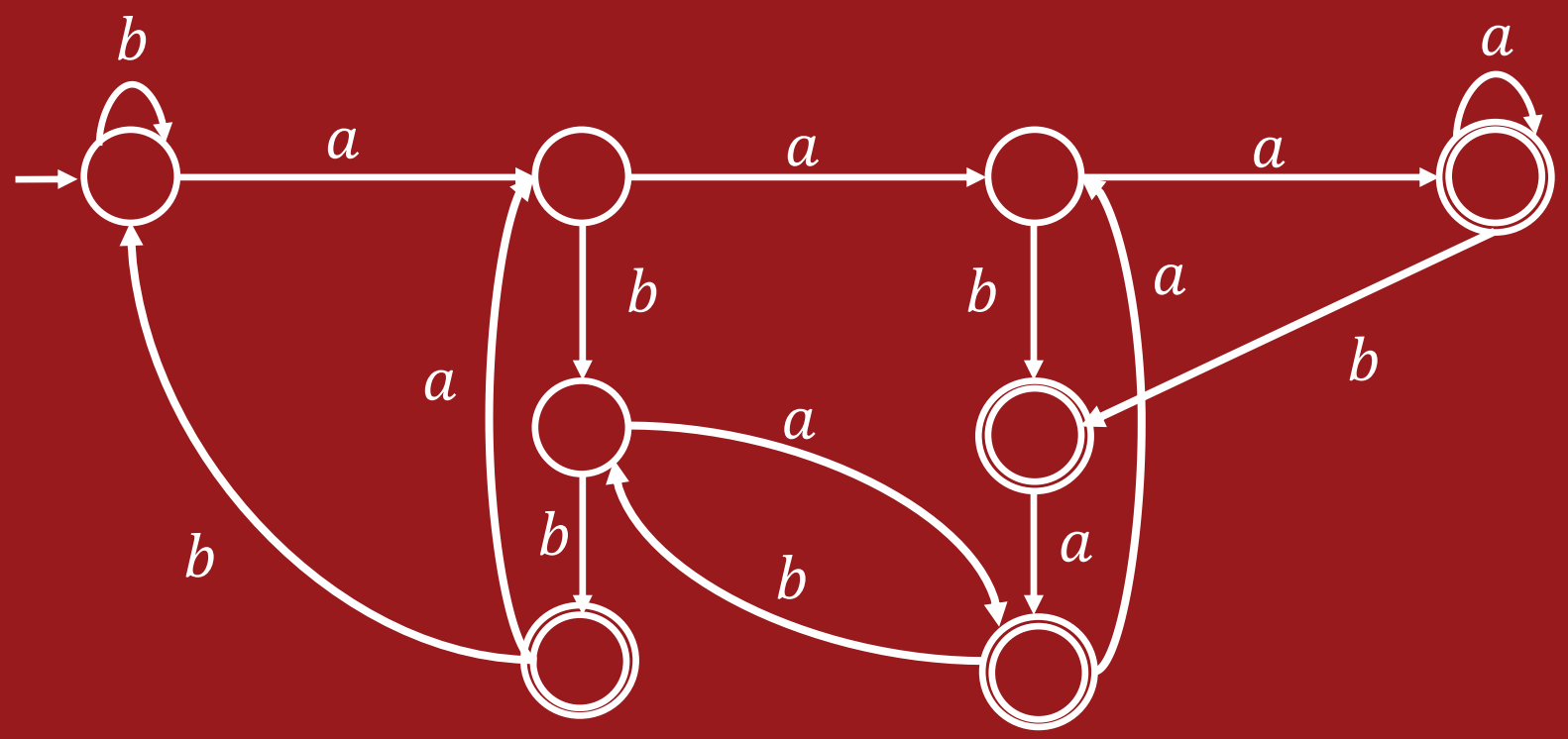


DFA A

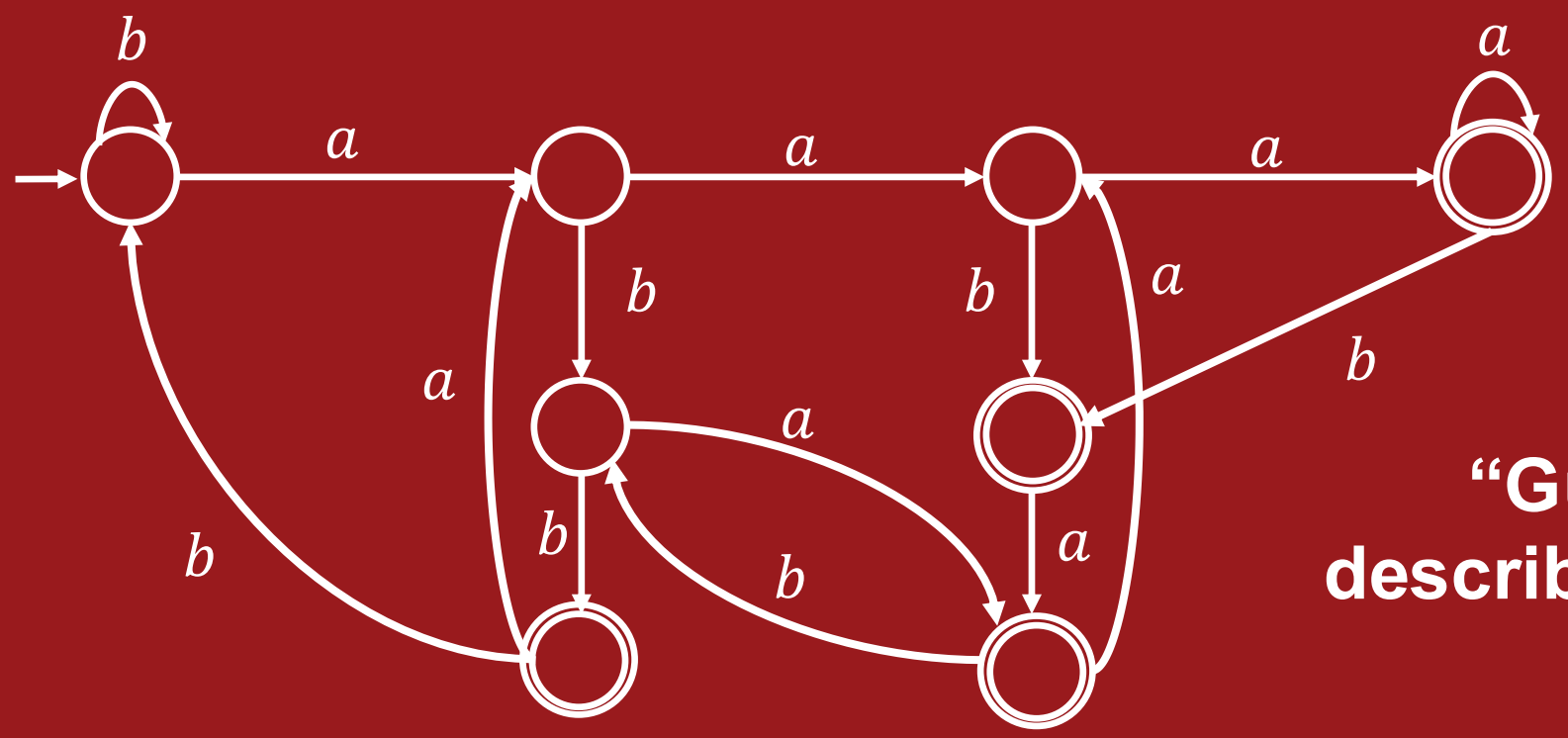


- Is the language empty? No: the word a is accepted (as we can reach a final state!)
- Is the word $ababa$ accepted by the automaton? Yes, there is a run that ends in a final state
- What is the language of A (or A)? „all words over $\{a, b\}$ that end with an a“ ➔ $\{a, b\}^* \{a\}$

Another example: guess the accepted language ;)



Another example: guess the accepted language ;)

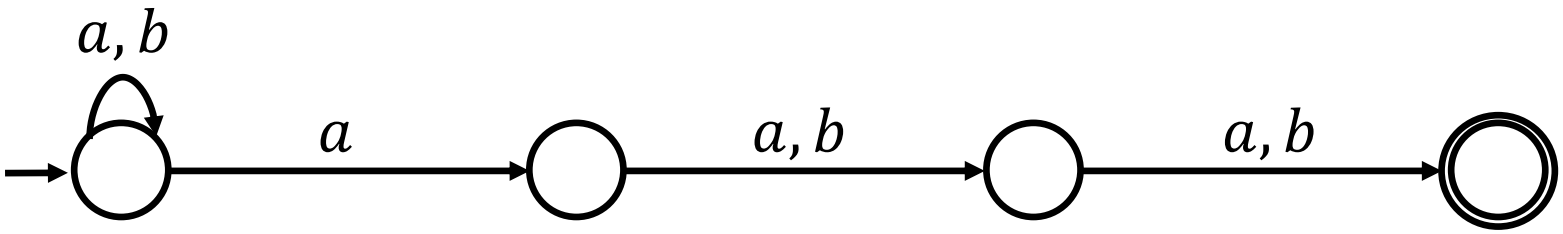


“Guessing” and describing the language is hard!

$L = \{a, b\}^* \, a \, \{a, b\}^2$

„the third to last symbol is an a“

An NFA for the same language



$L = \{ a, b\}^* \, a \, \{a, b\}^2$

„the third to last symbol is an a“

Two ways to describe a formal language

Automata (DFA, NFA)

- Easy to check **membership**
- We can construct automata for union, intersection, complement, …
- Verbose: might require many states, even for simple languages

Declarative (mathematics)

- Concise, often more readable
- More expressive than automata
- Unclear how to check membership

TODAY➔ Find mathematical notation for the languages accepted by finite automata

02141 – Computer Science Modelling

RL3 – REGULAR EXPRESSIONS

What are regular expressions?

“concise algebraic notation for regular languages”

Example

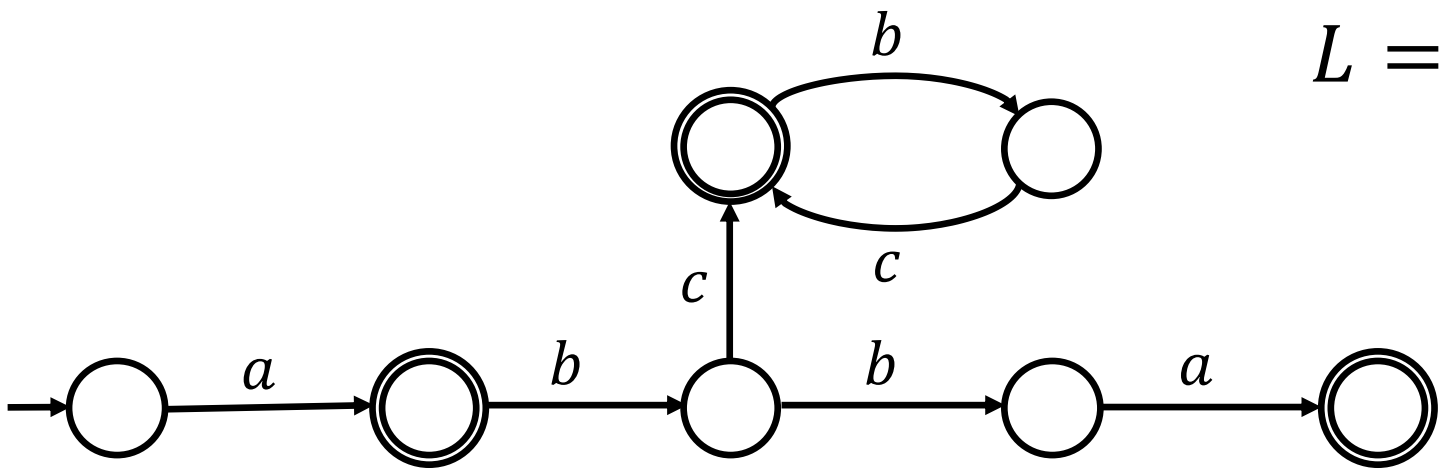
language union

Kleene closure

$$abba + a \cdot (bc)^*$$

every word is a regular expression

language concatenation



$$L = \{ abba \} \cup \{a\} \cdot \bigcup_{k \geq 0} \{bc\}^k$$

Syntax

language union

Kleene closure

$$abba + a \cdot (bc)^*$$

every word is a regular expression

language concatenation

Definition: The set *RegEx* of **regular expressions** E is given by:

$$E \ ::= \ \emptyset \mid \varepsilon \mid a \mid E_1 + E_2 \mid E_1 \cdot E_2 \mid E^*$$

symbol in Σ

$\cdot$ is typically omitted

Order of precedence (when you don't have parentheses): $* > \cdot > +$

Languages of Regular Expressions

The **language** of a regular expression E is $L(E) \in P(\Sigma^*)$

We can define it recursively as follows:

$\boldsymbol{E}$	$\boldsymbol{L(E)}$
$\emptyset$	$\emptyset$
ε	$\{\varepsilon\}$
a	$\{a\}$

$\boldsymbol{E}$	$\boldsymbol{L(E)}$
$E_1 + E_2$	$L(E_1) \cup L(E_2)$
$E_1 \cdot E_2$	$L(E_1)\ L(E_2)$
E_1^*	$L(E_1)^*$

Examples from Compilers

1. Keywords in programming languages

```
K  =  if  +  else  +  while  +  do

L(K) = { if, else, while, do }
```

2. Identifiers in programming languages

```
id  =  alpha (alpha + digit)*, where

- alpha  =  a + b + c + … + z + A + B + C + … + Z

- digit  =  0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9
```

`L(id)` describes all alpha-numeric sequences that start with a letter

Regular Expressions in Practice

- Unix tools: grep, awk, perl, …
- Text editors: *VSCode*, vim, emacs, …
- Compilers: define keywords, names, …

- All of these tools use similar syntax with lots of additional syntactic sugar

Our RegEx.	Extended RegEx.
a + b	a | b
a + ε	a?
a a*	a+
a+b+c	[abc] [a-c]

more on **regular-expressions.info**

Solution

Find a regular expression that matches all words on the left-hand side and none of the words on the right-hand side.

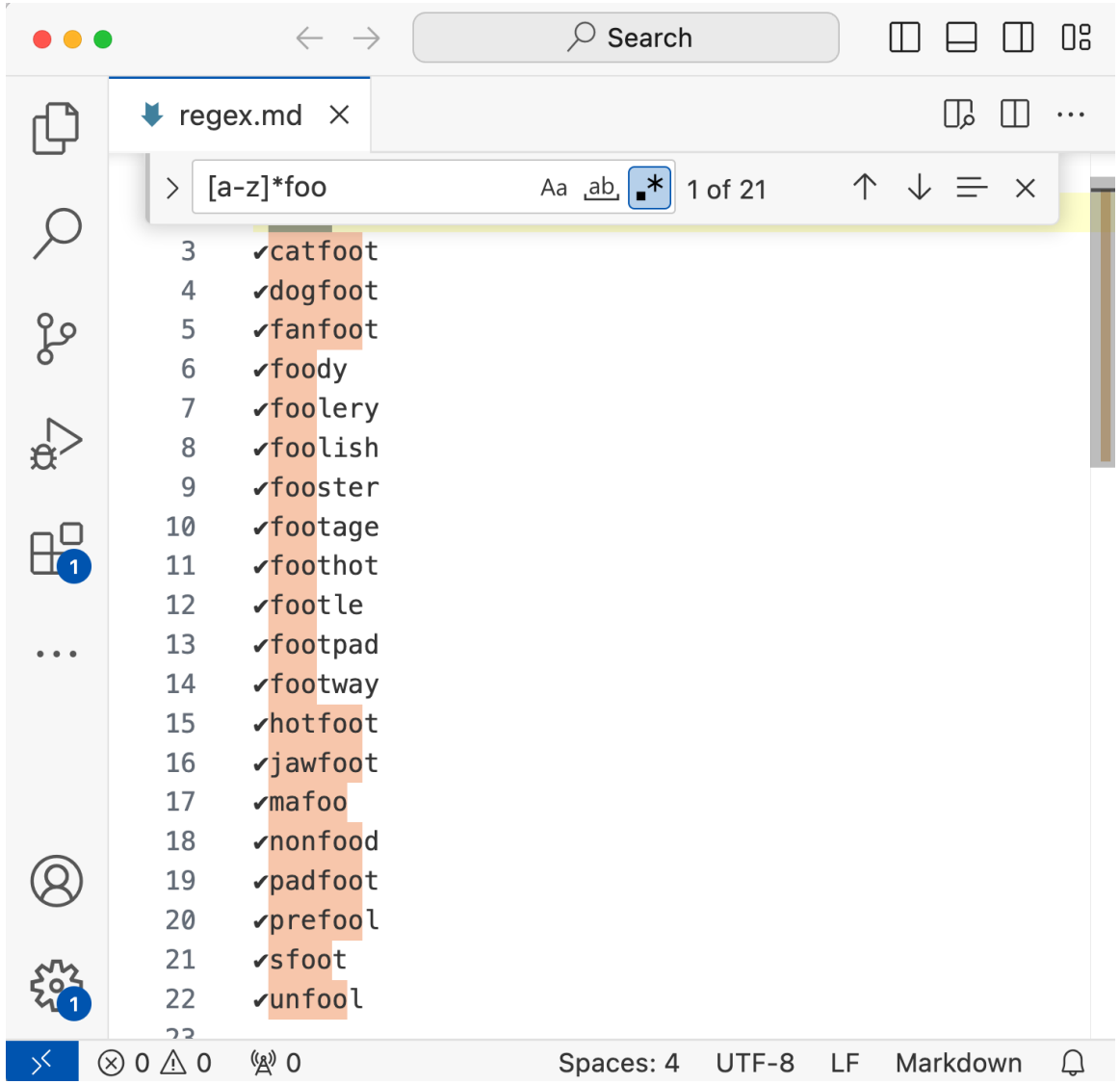
(a+…+z)*foo(a+…+z)*

or

[a-z]*foo[a-z]*

Match all of these…	and none of these…
✔ afoot	✘ Atlas
✔ catfoot	✘ Aymoro
✔ dogfoot	✘ Iberic
✔ fanfoot	✘ Mahran
✔ foody	✘ Ormazd
✔ foolery	✘ Silipan
✔ foolish	✘ altared
✔ fooster	✘ chandoo
✔ footage	✘ crenel
✔ foothot	✘ crooked
✔ footle	✘ fardo
✔ footpad	✘ folksy
✔ footway	✘ forest
✔ hotfoot	✘ hebamic
✔ jawfoot	✘ idgah
✔ mafoo	✘ manlike
✔ nonfood	✘ marly
✔ padfoot	✘ palazzi
✔ prefool	✘ sixfold
✔ sfoot	✘ tarrock
✔ unfool	✘ unfold

Regular expressions in VSCode



Exercise: name the languages described by the following regex?

1. $(a + b)^* \, a \, (a+b) \, (a+b)$

2. $(a + b)^* \, (a + bb)$

3. $(a+b)^* \, aa \, (a+b)^*$

Exercise: name the languages described by the following regex?

1. *(a + b)* a (a+b) (a+b)*

   ”all words over {a,b} where the third to last symbol is an a”

2. *(a + b)* (a + bb)*

   “all words over {a,b} that end with an a or with bb”

3. *(a+b)* aa (a+b)**

   “all words over {a,b} that contain aa as an infix.”

Open Questions:

Can every language accepted by an automaton also be described by a regular expression?

Can every language described by a regular expression also be accepted by a finite automaton?

Yes!

that's why we say DFAs accept *regular languages*

From Regular Expressions to Finite Automata

Main idea: construct an ε-NFA $A(E)$ for every regular expression E

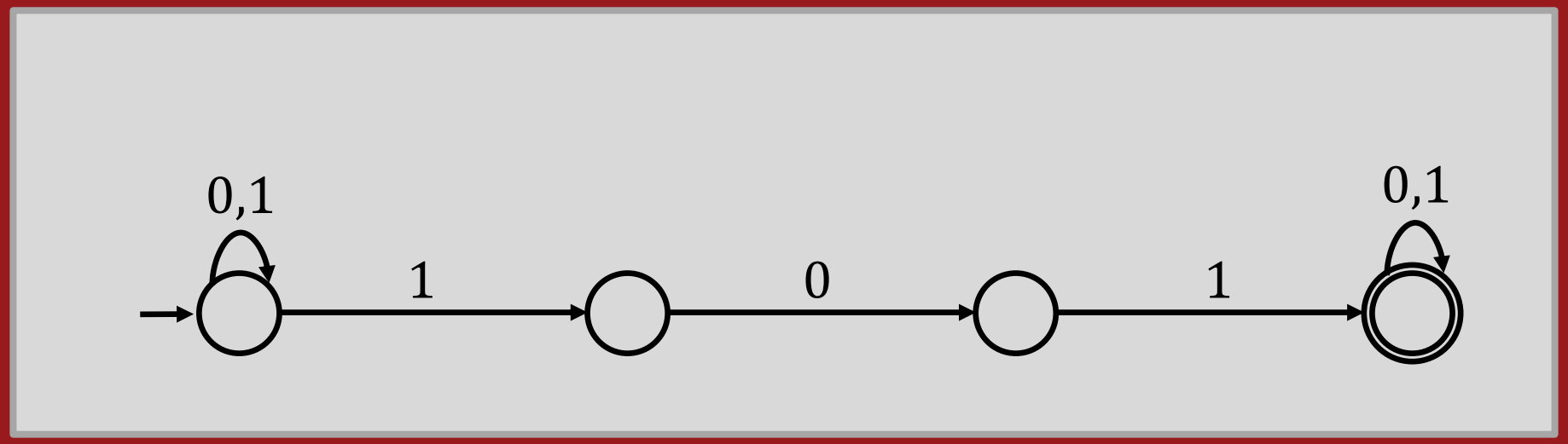
“Thompson's construction”

Exercise

Try to construct an NFA for the regular expression $(0+1)^* \, 101 \, (0+1)^*$

Solution

Try to construct an NFA for the regular expression (0+1)* 101 (0+1)*



➔ How do we construct such automata algorithmically?

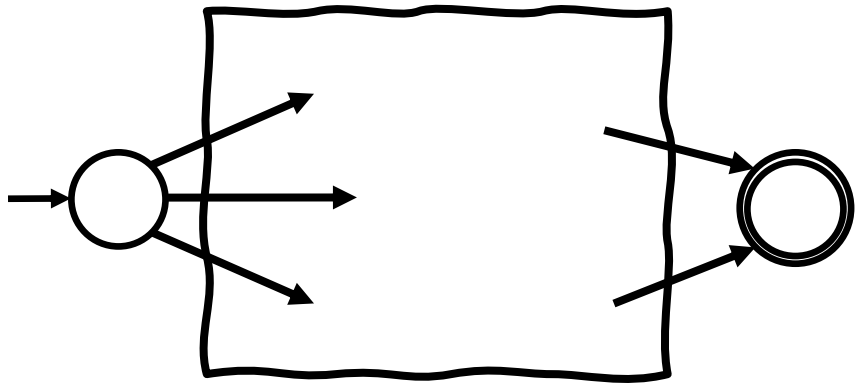
From Regular Expressions to ε−NFAs

We introduce a function A: $RegEx \rightarrow \varepsilon$−NFAs that can be defined recursively on the structure of regular expressions.

It allows us to give **automata-based semantics** to regex.

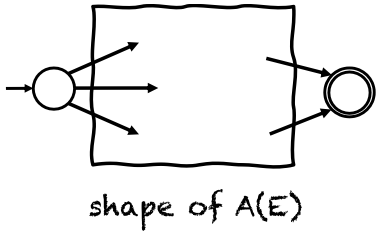
We construct the ε-NFA $A(E)$ such that it maintains three properties:

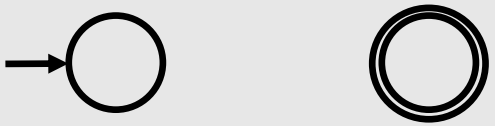
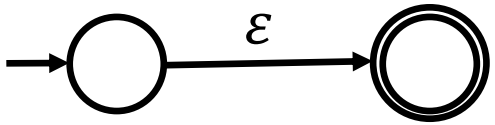
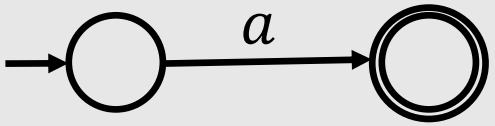
1. $A(E)$ has **exactly one final state**
2. $A(E)$ has **no transitions that enter the initial state**
3. $A(E)$ has **no transitions that leave the final state**

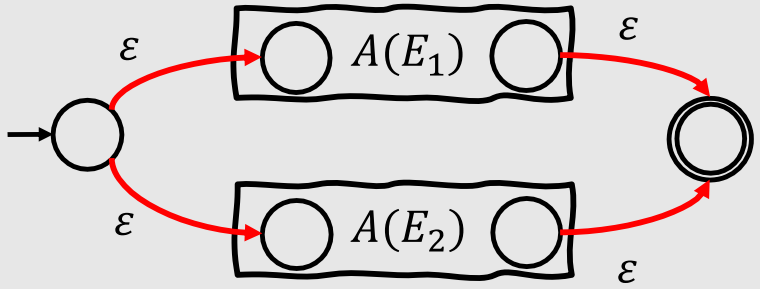
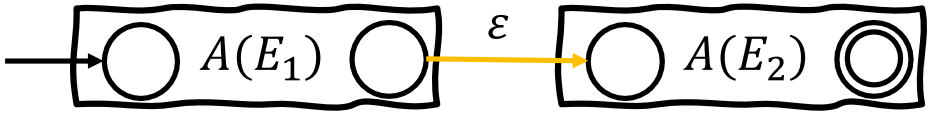
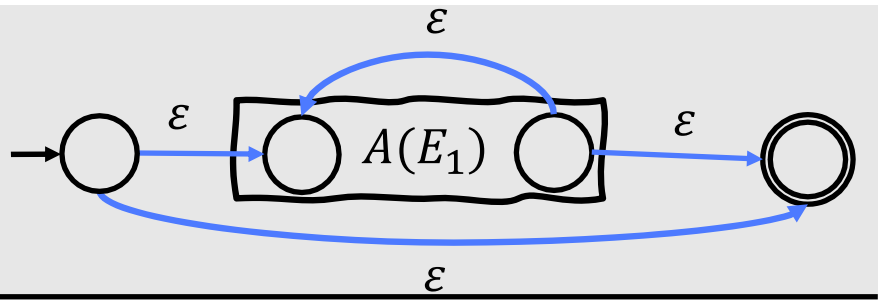


shape of A(E)

Automata-Based Semantics of Regular Expressions

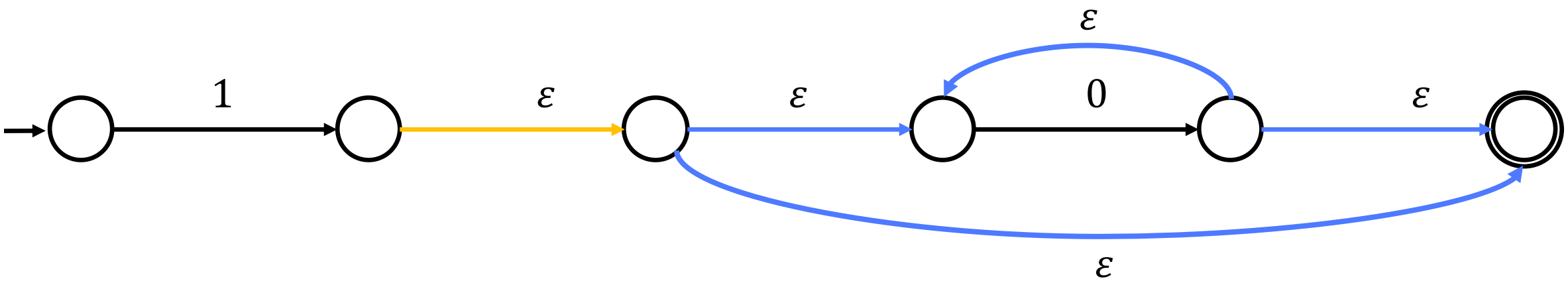


E	$A(E)$
$\emptyset$	
ε	
a	

E	$A(E)$
$E_1 + E_2$	
$E_1 \cdot E_2$	
E_1^*	

Example: an automaton for 1 0*

Example: an automaton for 1 0*



Regular expressions accept regular languages

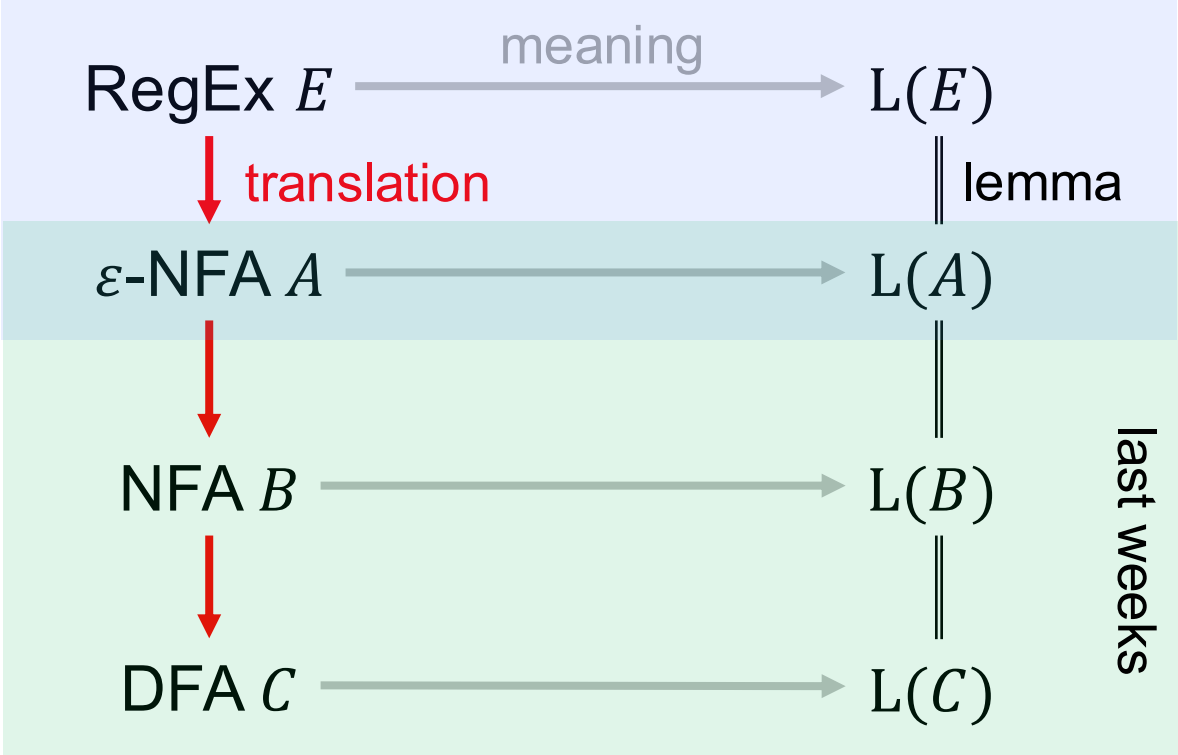
Lemma (semantic equivalence).

For every regular expression E, we have $L(E) \;=\; L(\,A(E)\,)$.

Proof. By structural induction on the syntax of E. (on the board)

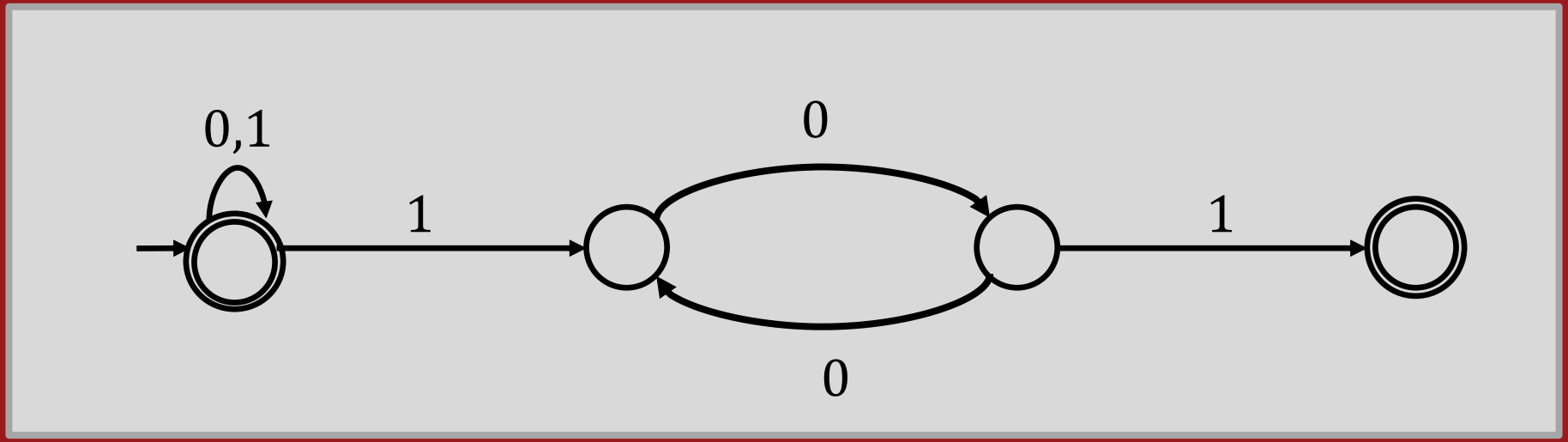
Theorem.

For every regular expression E, there exists a DFA C such that $L(E) = L(C)$.

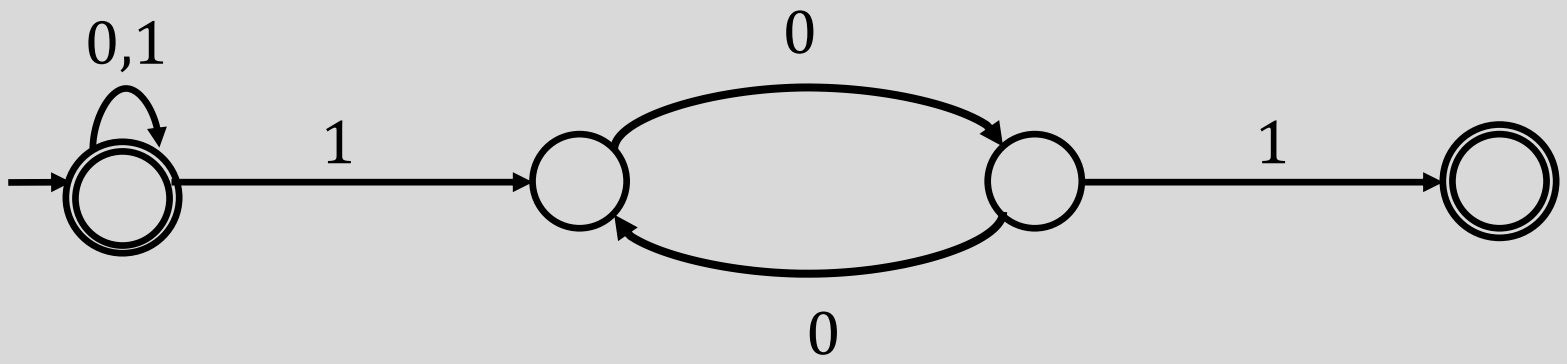


From Finite Automata to Regular Expressions

Exercise: Try to construct a RegEx for the following NFA



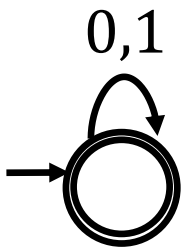
Solution: Try to construct a RegEx for the following NFA



(0+1)* + (0+1)* 1 0 (00)* 1

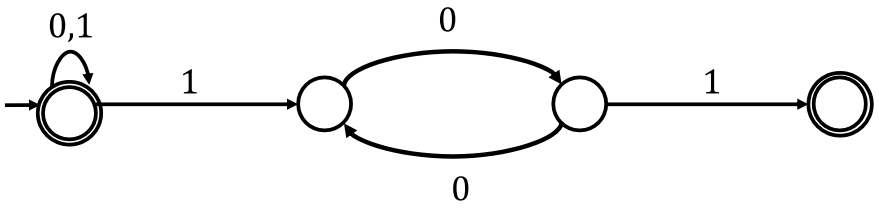
Insights

1. For small NFAs with one final state, we can construct a RegEx according to a pattern.



(0+1)*

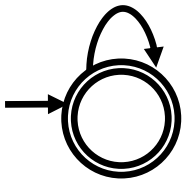
2. We can treat multiple final states as a **sum** of NFAs with one final state.



(0+1)* + (0+1)* 1 0 (00)* 1

3. If the alphabet consists of regular expressions, then the RegEx obtained from a pattern is a RegEx over the symbols in the "RegEx alphabet".

$a\ b^*\ a$

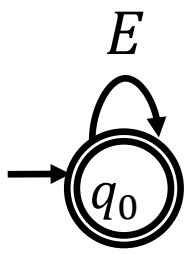


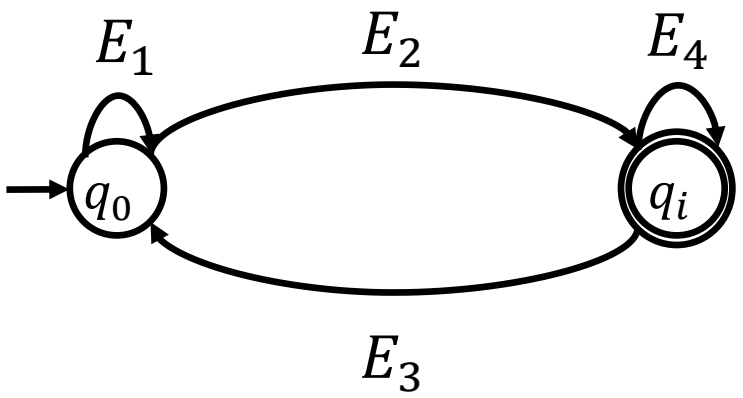
$(a\ b^*\ a)^*$

➔ **Idea**: reduce NFAs to known patterns by eliminating states and labeling edges with RegEx's.

NFA to RegEx: Fixed Patterns

Our goal: arrive to one of such patterns!

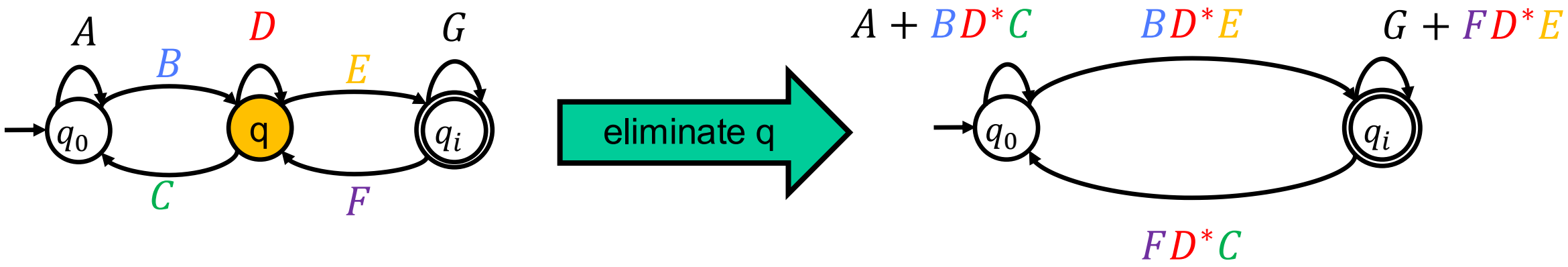




E^*

$E_1^* \; E_2 \; (E_4 + E_3 \; E_1^* \; E_2)^*$

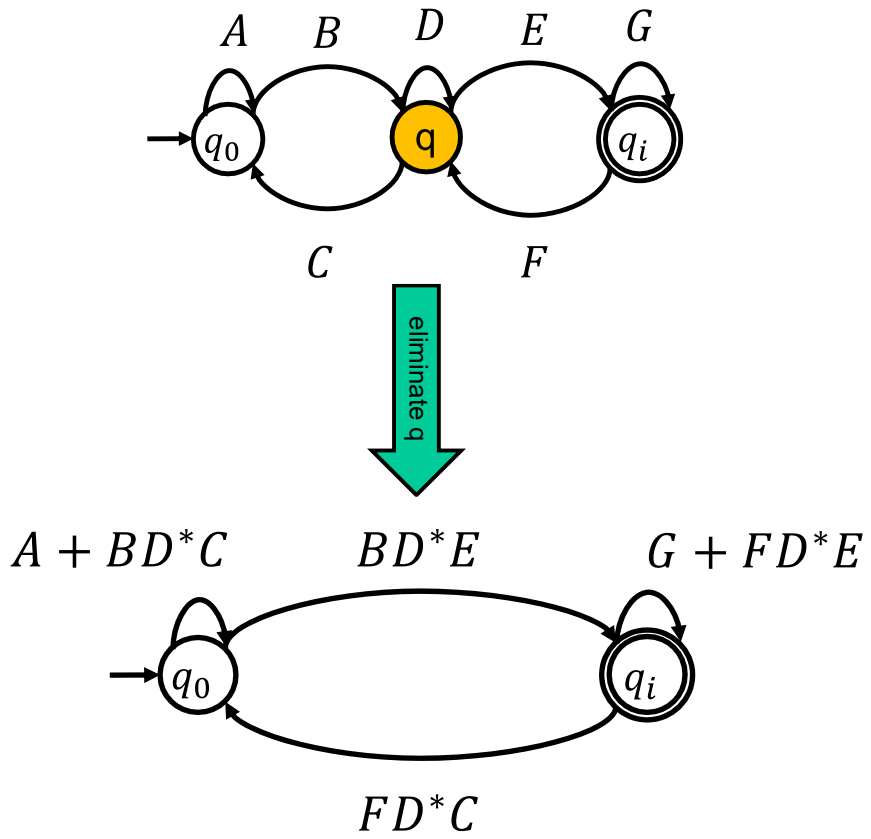
NFA to RegEx: State Elimination



NFA to RegEx: Algorithm

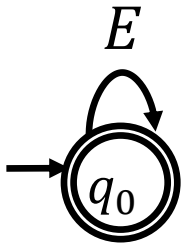
Input: NFA A with initial state q_0 and final states F.

Output: regular expression E such that $L(E) = L(A)$.

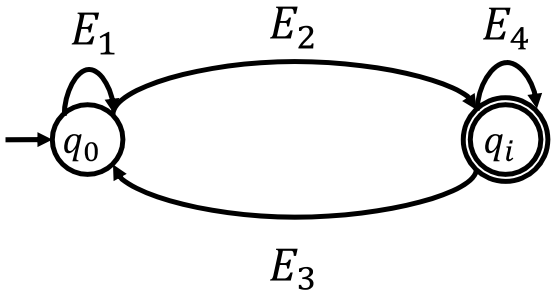


1. For every final state $q_i \in F$:
   i. Eliminate all states of A except q_0 and q_i

   ii. Derive a regular expression R_i by pattern matching



$R_i = E^*$

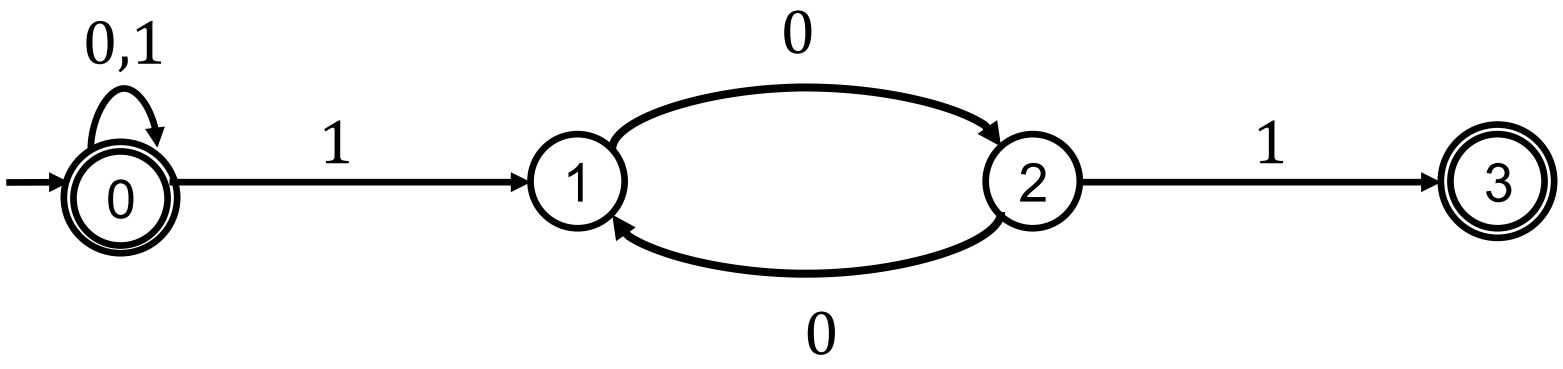


$R_i = E_1^* \; E_2 \; (E_4 + E_3 \; E_1^* \; E_2)^*$

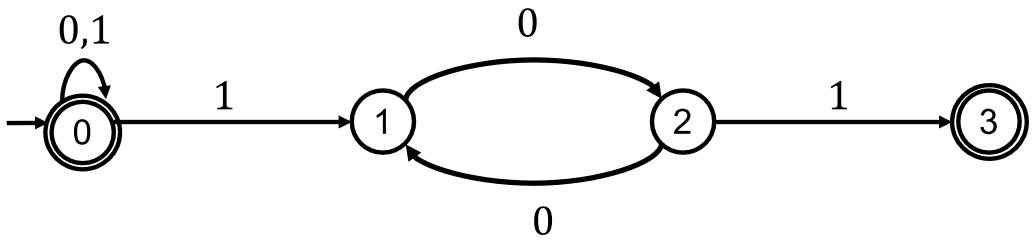
2. Return the sum of all derived regular expressions

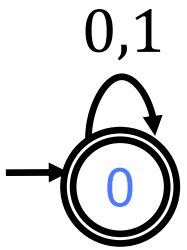
$E = \sum_{i \; s.t. \; q_i \in F} R_i$

Example

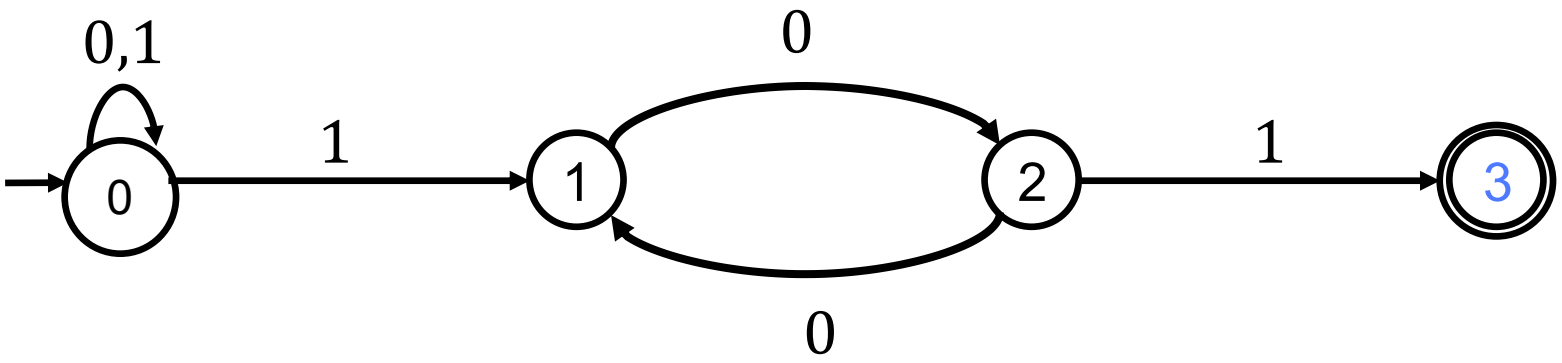


Example

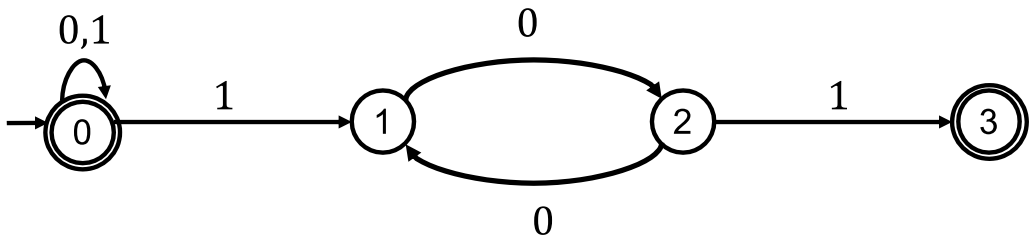


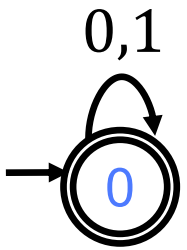


$R_0 = (0 + 1)^*$

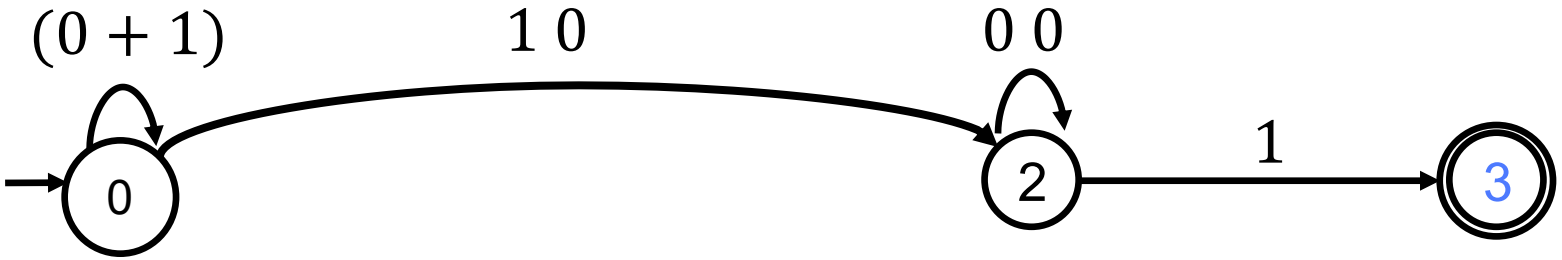


Example

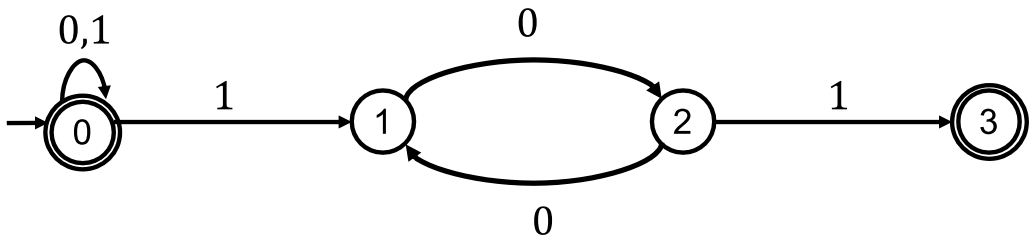


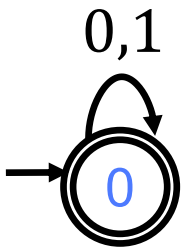


$R_0 = (0 + 1)^*$

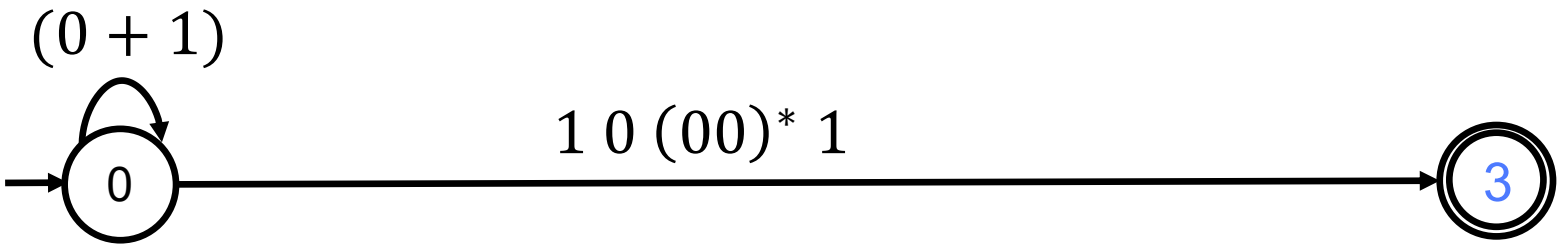


Example





$$R_0 = (0+1)^*$$



$$R_3 = (0+1)^*\ 1\ 0\ (00)^*1$$

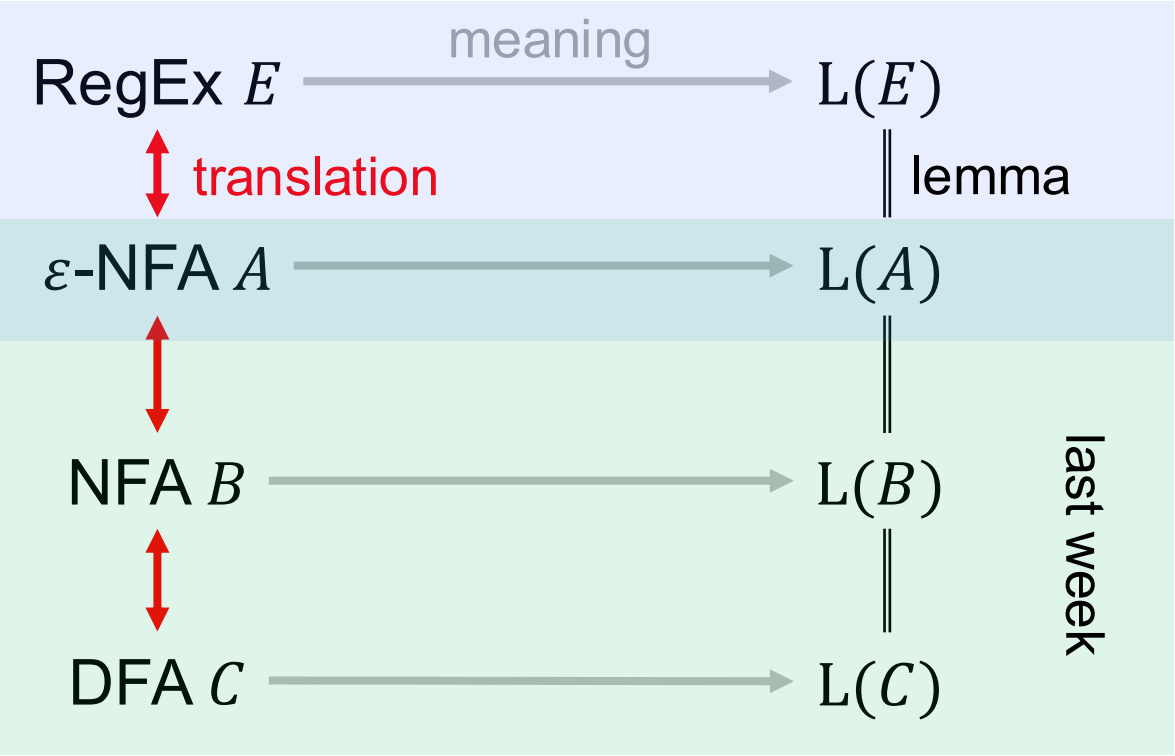
$$E \;=\; R_0 + R_3 \;=\; (0+1)^* + (0+1)^*\ 1\ 0\ (00)^*1$$

Summary

- Regular expressions and DFAs accept exactly the class of regular languages

RegEx ➔ DFA via Thompson's construction

DFA ➔ RegEx via state elimination



Quiz

1. Given two regular expressions E_1 and E_2, is there always a regular expression E such that $L(E) = L(E_1) \cap L(E_2)$?

2. Can we check whether all words accepted by a DFA A are also in the language of a regular expression E?

3. Is there a regular expression that is significantly smaller than any DFA accepting the same language?

Solution

1. Given two regular expressions E_1 and E_2, is there always a regular expression E such that $L(E) = L(E_1) \cap L(E_2)$?

   Yes, because DFAs are closed under intersection and we can translate back and forth between regular expressions and DFAs.

2. Can we check whether all words accepted by a DFA A are also in the language of a regular expression E?

   Yes, we can transform E into a DFA B and check whether $L(A) \backslash L(B) = \emptyset$

3. Is there a regular expression that is significantly smaller than any DFA accepting the same language?

   Yes, (a+b) a (a+b) … (a+b) (n times) can only be accepted by a DFA with at least 2^n states

Properties of Regular Expressions

Equivalence of Regular Expressions

Definition: Regular expressions are **equivalent** if they have the same language

$$E_1 = E_2 \quad \text{if and only if} \quad L(E_1) = L(E_2)$$

$$\left(\left(0\,(1+\varepsilon)\right)^* + (0+01)^*\right)^* = (01+0)^* \ ?$$

- Approach 1: transform both into DFAs and check equivalence
- Approach 2: simplify regular expressions using calculation rules

„language union and concatenation behave similarly to addition and multiplication“

Algebraic Laws of Regular Expressions

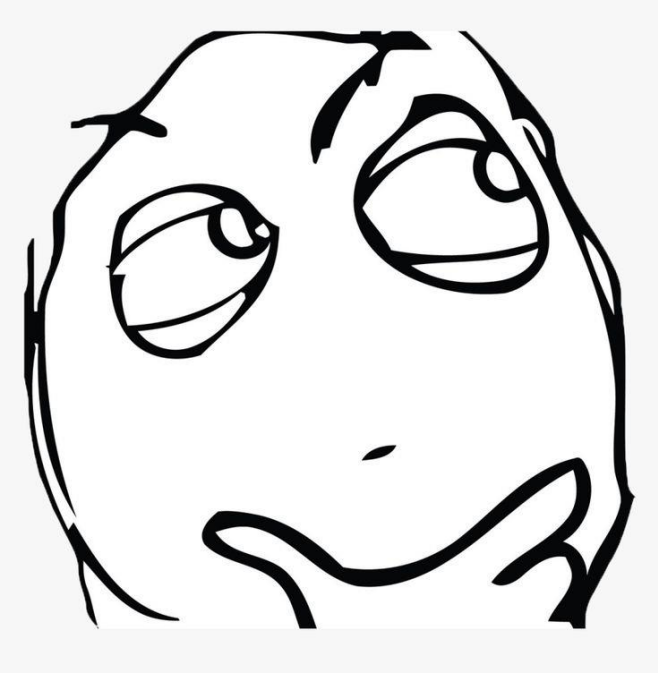
Examples:

- Distributivity law: $X\,(Y + Z) = \; X\,Y + X\,Z$
- Commutativity law for union: $X + Y \; = Y + X$
- Identity law for concatenation: $X \; \cdot \varepsilon \; = \varepsilon \cdot X \; = \; X$

More laws can be obtained. How to check? Remember that $E_1 = E_2$ iff $L(E_1) = L(E_2)$. Replace E_1 and E_2 with concrete expressions and test the language equivalence.

Algebraic Laws of Regular Expressions

But actually, let us get all(?) these laws now…



Commutative and associative laws

- Commutative law for **union**: L+M = M+L
- Associative law for **union**: (L + M) + N = L + (M + N)
- Associative law for **concatenation**: (L M) N = L (M N)
- Commutative law for **concatenation** **does not hold**: L M ≠ M L

Identity and annihilator laws

- Identity for **union**: $\emptyset + L = L + \emptyset = L$
- Identity for **concatenation**: $\varepsilon L = L \varepsilon = L$
- Annihilator for **concatenation**: $\emptyset L = L \emptyset = \emptyset$

Distributive and idempotent laws

- Left distributive law: $L (M + N) = L M + L N$
- Right distributive law: $(M + N) L = M L + N L$
- Idempotent law for **union**: $L + L = L$

Laws for closure operator

- $(L^*)^* = L^*$
- $\emptyset^* = \varepsilon$
- $\varepsilon^* = \varepsilon$
- $L+ = L\ L^* = L^*\ L$
- $L^* = L+ + \varepsilon$
- $L? = L + \varepsilon$

Quiz: Determine which rules are laws for regular expressions

1. Associativity for addition: $(X + Y) + Z = X + (Y + Z)$

2. Commutativity for concatenation: $X\,Y = Y\,X$

3. $(X^*)^* = X^*$

4. $\emptyset^* = \varepsilon$

Quiz: Determine which rules are laws for regular expressions

1. Associativity for addition: (X + Y) + Z = X + (Y + Z)

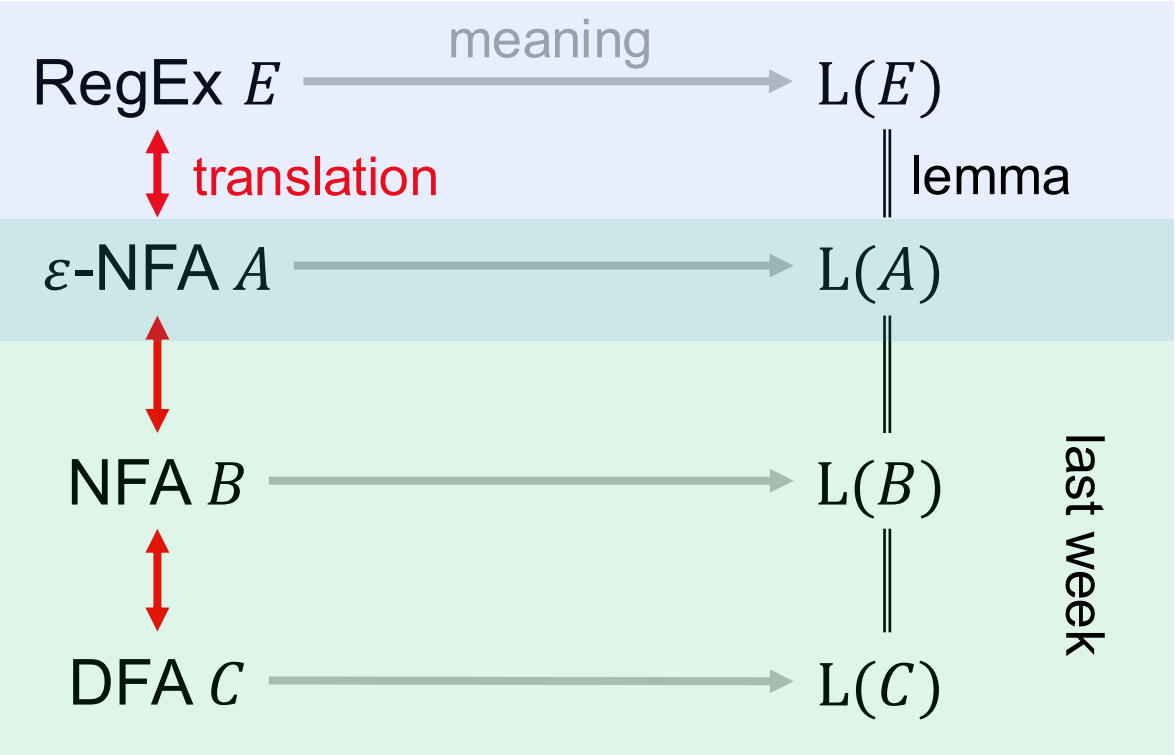
   $L\big((X+Y)+Z\big) = \big(L(X) \cup L(Y)\big) \cup L(Z) = L(X) \cup (L(Y) \cup L(Z))$

2. Commutativity for concatenation: X Y = Y X

   Counterexample: $a\,b \neq b\,a$

3. (X*)* = X*

4. $\emptyset^* = \varepsilon$

Wrap-up: Regular Expressions

- Regular expressions and DFAs accept exactly the class of regular languages
  - RegEx ➔ DFA via Thompson's construction
  - DFA ➔ RegEx via state elimination
- Regular expressions are *declarative*
  - Often concise descriptions of languages
  - Hard to solve problems like membership
  - Enjoy many algebraic properties of + and ·
- DFAs are *operational*
  - Easy to solve problems like membership
  - DFAs can be exponentially larger than regular expressions for the same language